commit-push-all

scripts/commit-push-all.sh — §11.4.234 dedicated commit/push entrypoint

Revision: 1 **Last modified:** 2026-08-18T18:49:45Z **Status:** active **Item:** task #66 (BOB-068 sweep-pattern interim remedy)

Overview

`scripts/commit-push-all.sh` is boba's single dedicated commit/push entrypoint per §11.4.234 — the always-unblocked mechanism through which routine work gets committed and pushed to every configured upstream. It runs, in order: fetch all remotes, cheap always-on validation, an optional/skippable long gate (`scripts/pre_build_verification.sh`), commit, push to all upstreams, and a closing summary.

As of task #66 it supports two commit modes:

- **Default (unscoped) mode** — `git add -A`. Stages and commits *everything* currently modified/untracked in the working tree. This is the original, backwards-compatible behavior.
- **Scoped mode** (`--scope <path>`, repeatable) — stages *only* the declared path(s), and REJECTS the commit if anything outside the declared scope ends up staged.

Why scoped mode exists — the BOB-068 pattern

Discovered 5 times in one session: because the default mode runs an **unconditional `git add -A`**, invoking `commit-push-all.sh` while a parallel subagent has an in-flight, not-yet-committed file elsewhere in the same working tree sweeps that unrelated file into the commit — a real §11.4.84 (working-tree quiescence) violation. Multiple subagents (BOB-064, BOB-067, BOB-069, BOB-075-fix, a13d1f01) reported either having their work swept, or having to work around the pattern by hand-crafting `git add` + `git commit` themselves (bypassing the dedicated entrypoint's validation stages entirely — itself a §11.4.234 regression). A reviewer flagged this as a Nit in the BOB-074 review.

`--scope` is the **interim tooling fix**: it lets a caller opt into a scoped `git add` instead of the sweep, backed by a safety check that catches the exact hazard `--scope` exists to prevent (a stray/ concurrent staged file). The full architectural remedy — §11.4.179-style isolated-per-stream git checkouts (each parallel subagent owning its own `.git`, so there is nothing to sweep) — is tracked as task #67 (proposal drafted, not yet built). `--scope` does not replace that remedy; it closes the gap while it's designed.

Prerequisites

- `bash`, standard POSIX utilities (`awk`, `sed`, `grep`), `git`.
- `flock` (optional — concurrent-run protection degrades to a WARN when absent, never blocks the mechanism per §11.4.234(B)).
- Invoked from anywhere inside the target git working tree; the script resolves `REPO_ROOT` via `git rev-parse --show-toplevel` and `cd`s there before doing any git work.

Usage examples

Default (unscoped) — unchanged, backwards-compatible

```
bash scripts/commit-push-all.sh "fix(proxy): retry on 429"
```

Stages everything (`git add -A`), commits, pushes to every configured remote. This is the existing behavior every current caller already relies on — nothing about it changed.

Scoped — new in task #66, preferred for parallel/ subagent work

```
bash scripts/commit-push-all.sh \
  --scope scripts/commit-push-all.sh \
  --scope docs/scripts/commit-push-all.md \
  --scope challenges/scripts/commit_push_all_scope_challenge.sh \

        "feat(scripts,#66): commit-push-all.sh --scope flag ends
        BOB-068 sweep pattern"
```

Only the three declared paths are `git add`-ed. If `git status` reveals anything else already staged when the safety check runs (see below), the whole commit is **rejected** — nothing is committed, the unexpected file(s) are named on stderr, and the working tree is left exactly as it was (the declared-scope files remain staged; nothing is force-unstaged, so a concurrent process's own staged work is not disturbed).

`--scope=<path>` (=-joined) is also accepted. `--scope` may be repeated any number of times for a multi-file commit.

Safety layer (the reviewer's exact concern)

Scoped mode does, in order:

1. `git add -- <scope-path-1> <scope-path-2> ...` — stage *only* the declared paths. Never `git add -A`.
2. Read the resulting staged set (`git diff --cached --name-only`).
3. For every staged path, confirm it **equals** a declared scope entry or sits **under** one (a declared scope entry may be a directory prefix). This is a structural match, never a substring match (§11.4.201(7)(a)).
4. If anything staged falls outside the declared scope — e.g. a concurrent subagent's own `git add`, or residue from an earlier unscoped `git add -A` that never got committed — the commit is **REJECTED** (exit 1) with every unexpected path printed, plus a remediation pointer (`git restore --staged <file>`).
5. Only if the staged set is an exact match for the declared scope does `git commit` run.

This directly implements the reviewer's request: verify `git status --short` shows *only* files in the declared scope before committing, and reject with an actionable message if extra files appear staged.

Backwards compatibility

Existing callers that invoke `commit-push-all.sh "message"` with no `--scope` flag are **completely unaffected** — the default path is byte-for-byte the same `git add -A` → commit → push flow that existed before task #66. `--scope` is purely additive. The known BOB-068 hazard remains present in default mode (that's the "known existing dependency" the task brief accepts) — new dispatches are the ones expected to move to `--scope`.

Guidance for SDD / subagent task dispatches

Going forward, task briefs that dispatch parallel subagents with a declared, disjoint file scope (the normal SDD dispatch shape) SHOULD have their subagent's final commit go through `commit-push-all.sh --scope <declared-scope-paths>` rather than the bare unscoped form — this is exactly the case `--scope` was built for, and it turns the brief's own "disjoint scope" declaration into a *mechanically enforced* commit boundary instead of a convention the subagent has to remember and honor by hand. Unscoped `commit-push-all.sh "message"` remains correct for genuinely whole-tree changes (e.g. a doc-sync regeneration sweep) where "everything currently modified" really is the intended scope.

Environment knobs

Variable	Effect
`BOBA_SYNC_SKIP_CI=1`	Skip the long `pre_build_verification.sh` gate for this run. Recorded as a `[skip-ci]` marker in the commit message (§11.4.234(D) — the skip is never silent).
`BOBA_SYNC_SKIP_LONG=1`	Alias of `BOBA_SYNC_SKIP_CI` (mirrors Lava's `LAVA_SYNC_SKIP_CI` naming).

Exit codes

Code	Meaning
0	Commit + push completed (or nothing to commit).
1	Validation failure — cheap check, long gate, **or the `--scope` safety check** — remediation printed to stderr.
2	Invocation error (missing commit message, malformed `--scope`, unknown flag).
3	Another `commit-push-all.sh` invocation already holds the flock.

Edge cases

- **Scope path doesn't exist.** `git add -- <path>` fails immediately (`fatal: pathspec '...' did not match any files`); under `set -e` the script exits with git's own error — no silent no-op.
- **`--scope` with zero matching changes.** If the declared path has no working-tree delta, `git diff --cached --quiet` is true after `git add`, and the script logs "nothing to commit" and proceeds straight to the push stage (same as default mode).
- **Directory scope.** `--scope some/dir` stages the whole subtree under `some/dir`; the safety check's prefix match (`some/dir/...`) accepts any staged path under it.
- **Mixing scoped and unscoped semantics.** There is no flag to mix "some declared paths + also sweep everything else" — that would defeat the purpose. Use two separate invocations if genuinely needed.

Anti-bluff verification

`challenges/scripts/commit_push_all_scope_challenge.sh` proves the safety layer is real (not decorative) via §11.4.115 RED/GREEN polarity, run entirely against a throwaway sandbox git repo (zero remotes — never touches the real boba repo or pushes anywhere):

- **Step A (always runs):** a genuine single-scope commit succeeds cleanly and lands *exactly* the declared file.
- **Step B (`RED_MODE=1`, default):** reproduces the BOB-068 hazard — an out-of-scope file is already staged (simulating a concurrent subagent) before `--scope` runs — and confirms `--scope` rejects, names the file, and leaves no new commit.
- **`RED_MODE=0`:** Step A only, for a fast regression-guard rerun.

```
bash challenges/scripts/
        commit_push_all_scope_challenge.sh          # full
        RED+GREEN polarity
RED_MODE=0 bash challenges/scripts/
        commit_push_all_scope_challenge.sh  # GREEN-only
        regression guard
```

A §1.1 paired mutation (stripping the rejection block from a scratch copy of `commit-push-all.sh` and re-running Step B's exact hazard scenario against it) was performed during task #66's implementation and confirmed the mutated script silently commits the out-of-scope file — proving the check is load-bearing. See `.superpowers/sdd/task-66-report.md` for the pasted terminal evidence.

Related scripts

- `scripts/pre_build_verification.sh` — the long gate this entrypoint runs at stage 3.
- `challenges/scripts/commit_push_all_scope_challenge.sh` — anti-bluff proof for `--scope`.
- `docs/proposals/` — the §11.4.179 isolated-git-streams proposal (task #67), the architectural remedy `--scope` is an interim substitute for.